Chapter 10. Mass-storage structure. Full solutions

These original derivations correspond to the stable identifiers in the separate exercises PDF.

OS10-E01. Disk scheduling

1. FCFS follows 50→82→170→43→140→24→16→190. Its movements are 32,88,127,97,116,8,174, totaling 642 cylinders.
2. SSTF follows 50→43→24→16→82→140→170→190. The successive distances are 7,19,8,66,58,30,20, totaling 208. Direction is not a restriction on SSTF; each step selects the nearest remaining request.
3. In an online model, new requests near the current head can always beat a distant request. The finite list above completes, so it illustrates distance rather than proving an infinite starvation execution.

Validation criterion. Show the service sequence and the distance calculation under the stated model.

OS10-E02. RAID 5 and mirror groups

1. RAID 5 has approximately three disks of usable data capacity and one disk-equivalent of distributed parity. Any one whole-disk failure can be reconstructed under the assumed intact-survivor model.
2. Two two-disk mirrors together hold approximately two disks of data. Each group tolerates one member loss, including one failure in each group simultaneously. Losing both members of one group loses that group's data. A single four-copy RAID 1 would instead hold only one disk of data.
3. Rebuilding one mirror commonly reads its surviving peer; RAID 5 rebuild reconstructs from the remaining stripe members. Reliability still depends on independent failures, unreadable sectors, rebuild rate, metadata, and consistent writes. Capacity arithmetic alone cannot rank complete system reliability.

Validation criterion. Keep array organization, capacity, and tolerated failure patterns distinct.

Chapter 10. Mass-storage structure. Full solutions (continued)

OS10-E03. A stable write

1. It may promise only that the OS accepted the bytes into volatile state. Without a stronger contract, ordinary buffered-write success need not promise survival of immediate power loss.
2. Use the specified data and required metadata persistence protocol, including ordering and device-cache completion. The exact flush or transaction operation must actually reach the stated persistence domain before durable success is reported.
3. No. Copies that both remain volatile or are acknowledged too early can disappear together. Mirroring adds protection only within a defined write and failure model.

Validation criterion. Distinguish cache acceptance, device acknowledgment, and durable commit.

OS10-E04 RPM, IOPS, and response

1. Rotation 60000/7200=25/3 ms, mean 25/6. Transfer 64 KiB/(128 MiB/s)=125/256 ms. Service 5+25/6+125/256=7415/768≈9.65495 ms.
2. Response 15+service≈24.65495 ms. Without queue and overlap, serial IOPS 1000/service≈103.574. Queue wait is not media service and this reciprocal does not describe overlapping queues.
3. 4 KiB×1000=3.90625 MiB/s; 64 KiB×1000=62.5 MiB/s. Sequential 1 GiB's pure-transfer bound is separately 8 s.

OS10-E05 Flash mappings and WA

1. Program P4, make L1→P4, and mark P1 invalid. Copy L0:P0→P5, L2:P2→P6, L3:P3→P7. Make mappings/content safe before erasing B0. Final L0→5, L1→4, L2→6, L3→7.
2. One host page. Without discard, physical 1+3=4 programs, WA=4. Discarded L2 needs no copy, physical 1+2=3, WA=3. Exclude mapping/controller overhead and erase cost; keep host-page-program denominator fixed.
3. Discard removes logical liveness, not immediate or secure erase. Crash recovery must preserve new data/mappings before erasing the sole old live source. Normal-path WA is not a crash-consistency proof.

Chapter 10. Mass-storage structure. Full solutions (continued)

OS10-E06 LOOK, SCAN, and wrap cost

1. LOOK 53→37→14→65→67→98→122→124→183,39+169=208. SCAN includes 0 between 14 and 65,53+183=236. Calling 208 SCAN contradicts this endpoint requirement.
2. SSTF 53→65→67→37→14→98→122→124→183,12+2+30+23+84+24+2+59=236. Equal total is coincidental. LOOK 208 proves SSTF is not globally movement-minimal.
3. C-SCAN travels upward to 199, wraps 0, then 14/37, total 146+199+37=382. C-LOOK ends 183, wraps 14, then 37, total 130+169+23=322. Fixed finite batches cannot establish infinite starvation; SSTF's risk requires continuing nearby arrivals. Sweeping bounds also require bounded per-sweep service/admission.

OS10-E07 Parity, write holes, and independent equations

1. P=3C xor A5 xor 5A=C3; D1=C3 xor 3C xor 5A=A5. Pnew=C3 xor A5 xor A4=C2. Cold small update is 2 reads+2 writes, excluding log/cache.
2. Rebuild D2=C3 xor 3C xor A4=5B instead of 5A. Stale parity violates coherent redundancy; single-device tolerance does not imply safety after an unprotected write hole plus another loss.
3. Q=3C xor 57 xor 75=1E. Removing known D0 gives Pmissing=FF, Qmissing=22. (2+4)B=22 xor(2・FF)=22 xor E3=C1. inverse 6=7A, so B=7A・C1=5A, A=FF xor 5A=A5. Use carryless 11D reduction, not integer multiplication.
4. A+B=Pmissing and αA+βB=Qmissing give (α+β)B=Qmissing+αPmissing. Distinct α/β make α+β invertible, uniquely determining B then A. A duplicate XOR is dependent and cannot solve two unknowns.

Chapter 10. Mass-storage structure. Full solutions (continued)

OS10-E08 Two-copy crash prefixes

1. During C1's write, C1 can be old, new, or torn, but C2 retains valid v1. At least v1 is recoverable; a complete valid v2 may be selected. During C2's write, C1 is already durable v2, so recovery selects v2 rather than v1.
2. The body contains a four-byte big-endian generation, ASCII V1 or V2 followed by 502 zero bytes, then CRC32 over the first 508 bytes. CRCs are BF4CA2E8 and 4A112E80. Validate CRC first, then select the highest generation. CRC magnitude is not version order; detection of the injected tears is a model assumption, not collision-free authentication.
3. While one copy is overwritten, the other remains valid. C1's new version is flushed before C2 changes, preserving at least one valid copy inductively. Recovery keeps the selected good copy, repairs and flushes the other first, and therefore retains a good copy if repair crashes. Simultaneous overwrites, unreliable flush, or common loss breaks the proof's premises.
4. Before ACK, recovery may select old or complete new data. An absent reply does not prove an absent effect. A CPU barrier does not flush device caches; write order does not guarantee a write cannot tear. FUA/flush depend on the device contract. This single-record guarantee does not establish multi-record transaction atomicity.

OS10-E09 Topology, interfaces, and swap references

1. A+C breaks both 0+1 stripes, but 1+0 retains B/D, one member per pair. A+B leaves 0+1's complete CD stripe but destroys 1+0's AB pair. Failure sets depend on topology.
2. NAS's server maps file handle/offset to blocks. SAN's host filesystem maps to LBA before transport. A shared LUN alone has no distributed filesystem locking/coherence; two ordinary exclusive-host filesystems can corrupt metadata if writing concurrently.
3. No. P2 still needs 41. Scoped swap-PTE refs 2→1 after P1, then 0 after P2 reads; reclaim additionally requires no in-flight I/O/cache ownership. P1's 42 needs separate backing if required. These counts are not Linux swap_map encoding.
4. A start 2048, count 204800, exclusive end 206848; B start 206848, count 409600, exclusive end 616448. The ranges do not overlap. This is a teaching partition model, not complete GPT layout.

Chapter 10. Mass-storage structure. Full solutions (continued)

Verification code appendix: storage_models.py

```
"""Offline/serial storage teaching models. No real device I/O is performed."""
from fractions import Fraction
from functools import reduce
from operator import xor
import zlib


def movement(path):
    return sum(abs(right-left) for left,right in zip(path,path[1:]))


def schedule(head,requests,policy,maximum=199):
    if not 0<=head<=maximum or any(not 0<=item<=maximum for item in requests):
        raise ValueError("cylinder outside device")
    pending=list(requests)
    if policy=='FCFS':return [head]+pending
    if policy=='SSTF':
        path=[head]
        while pending:
            index=min(range(len(pending)),key=lambda i:(abs(pending[i]-
path[-1]),pending[i],i))
            path.append(pending.pop(index))
        return path
    prefix=[head]+[item for item in pending if item==head]
    lower=sorted(item for item in pending if item<head)
    upper=sorted(item for item in pending if item>head)
    if policy=='LOOK-down':return prefix+lower[::-1]+upper
    if policy=='SCAN-down':return prefix+lower[::-1]+([0] if upper else [])+upper
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 10. Mass-storage structure. Full solutions (continued)

```
    if policy=='C-LOOK-up':return prefix+upper+lower
    if policy=='C-SCAN-up':return prefix+upper+([maximum,0] if lower else [])+lower
    raise ValueError("unknown policy")


def service_ms():
    rotation=Fraction(60000,7200)/2
    transfer=Fraction(64*1024*1000,128*1024*1024)
    return Fraction(5)+rotation+transfer


def parity(values):
    return reduce(xor,values,0)


def gf_multiply(a,b):
    result=0
    while b:
        if b&1:result^=a
        a<<=1
        if a&0x100:a^=0x11D
        b>>=1
    return result


def gf_inverse(value):
    if not value:raise ValueError("zero has no inverse")
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter 10. Mass-storage structure. Full solutions (continued)

```
    result=1
    for _ in range(254):result=gf_multiply(result,value)
    return result


def recover_two(p_missing,q_missing,alpha,beta):
    if alpha==beta:raise ValueError("dependent parity equations")
    b=gf_multiply(gf_inverse(alpha^beta),q_missing^gf_multiply(alpha,p_missing))
    return p_missing^b,b


def record(generation):
    payload=f"V{generation}".encode()
    payload+=bytes(504-len(payload))
    body=generation.to_bytes(4,'big')+payload
    return body+zlib.crc32(body).to_bytes(4,'big')


def valid(data):
    return len(data)==512 and zlib.crc32(data[:508])==int.from_bytes(data[508:],'big')


def recover(copies):
    good=[item for item in copies if valid(item)]
    if not good:raise ValueError("no valid copy remains")
    return max(good,key=lambda item:int.from_bytes(item[:4],'big'))
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 10. Mass-storage structure. Full solutions (continued)

```
def flash_gc(trim_l2=False):
    mapping={0:0,1:1,2:2,3:3};trace=[dict(mapping)]
    mapping[1]=4;physical_programs=1;trace.append(dict(mapping))
    if trim_l2:del mapping[2]
    target=5
    for logical,physical in list(mapping.items()):
        if physical<4:
            mapping[logical]=target;target+=1;physical_programs+=1
    trace.append(dict(mapping))
    return {'mapping_trace':trace,'erased_pages':[0,1,2,3],
            'host_page_writes':1,'physical_page_programs':physical_programs}


if __name__=='__main__':
    requests=[98,183,37,122,14,124,65,67]
    for policy in ['FCFS','SSTF','LOOK-down','SCAN-down','C-SCAN-up','C-LOOK-up']:
        path=schedule(53,requests,policy)
        print(policy,path,movement(path))
    print('service_ms:',service_ms(),float(service_ms()))
    print('parity:',hex(parity([0x3C,0xA5,0x5A])))
    print('FTL:',flash_gc())
    for generation in (1,2):print('record',generation,record(generation)[508:].hex())
    print('recovered generation:',int.from_bytes(recover([record(2),record(1)])[:4],'big'))
```

References and attribution

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study